DevOps Interview Preparation

Phase 5 | December 2026 | 80+ Questions

Generated: August 2026

Linux | Git | Docker | Kubernetes | CI/CD | Terraform | Cloud | Scenarios

Interview Preparation DevOps Engineer

Phase: 5 (December 2026) | Start revising from Week 17

Preparation Strategy

Category	Weight	Focus
Technical (Tools & Concepts)	40%	Deep knowledge of DevOps stack
Scenario-Based	30%	"How would you handle...?"
System Design	15%	Architecture & scalability
Behavioral	15%	STAR method, leadership, teamwork

1. Linux & Shell Scripting (10 Questions)

#	Question	Confidence
1	Explain the Linux boot process	[ ] Low [ ] Med [ ] High
2	What is the difference between a process and a thread?	[ ] Low [ ] Med [ ] High
3	How do you troubleshoot a server with high CPU usage?	[ ] Low [ ] Med [ ] High
4	Explain file permissions and the chmod command	[ ] Low [ ] Med [ ] High
5	What are inodes? What happens when you run out?	[ ] Low [ ] Med [ ] High
6	Write a script to monitor disk usage and alert	[ ] Low [ ] Med [ ] High
7	What is the difference between soft and hard links?	[ ] Low [ ] Med [ ] High
8	How do you schedule tasks in Linux?	[ ] Low [ ] Med [ ] High
9	Explain systemd and how services are managed	[ ] Low [ ] Med [ ] High
10	What are namespaces and cgroups? (Container foundation)	[ ] Low [ ] Med [ ] High

2. Git & Version Control (10 Questions)

#	Question	Confidence
1	Explain the difference between git merge and git rebase	[ ] Low [ ] Med [ ] High
2	How do you resolve a merge conflict? Walk through it	[ ] Low [ ] Med [ ] High
3	What branching strategy does your team follow?	[ ] Low [ ] Med [ ] High
4	What is git stash and when would you use it?	[ ] Low [ ] Med [ ] High
5	How do you revert a commit that's already pushed?	[ ] Low [ ] Med [ ] High
6	Explain git cherry-pick with a use case	[ ] Low [ ] Med [ ] High
7	What is the difference between git reset and git revert?	[ ] Low [ ] Med [ ] High
8	How do you set up branch protection rules?	[ ] Low [ ] Med [ ] High
9	What are Git hooks? Give 2 practical examples	[ ] Low [ ] Med [ ] High
10	How do you handle large files in Git?	[ ] Low [ ] Med [ ] High

3. Docker (10 Questions)

#	Question	Confidence
1	What is the difference between a container and a VM?	[ ] Low [ ] Med [ ] High
2	Explain Docker's layered architecture	[ ] Low [ ] Med [ ] High
3	What is the difference between CMD and ENTRYPOINT?	[ ] Low [ ] Med [ ] High
4	How do you optimize a Docker image for production?	[ ] Low [ ] Med [ ] High
5	Explain Docker networking (bridge, host, overlay)	[ ] Low [ ] Med [ ] High
6	How do you handle persistent data in Docker?	[ ] Low [ ] Med [ ] High
7	What are multi-stage builds and why are they important?	[ ] Low [ ] Med [ ] High
8	How do you scan Docker images for vulnerabilities?	[ ] Low [ ] Med [ ] High

9

What is Docker Compose and its use case?

[ ] Low [ ] Med [ ] High

10

How do you debug a container that keeps crashing?

[ ] Low [ ] Med [ ] High

4. Kubernetes (15 Questions)

#	Question	Confidence
1	Explain Kubernetes architecture and components	[] Low [] Med [] High
2	What is the difference between a Deployment and a StatefulSet?	[] Low [] Med [] High
3	How does a Service discover backend Pods?	[] Low [] Med [] High
4	Explain the different Service types and when to use each	[] Low [] Med [] High
5	What is an Ingress Controller and how does it work?	[] Low [] Med [] High
6	Explain RBAC in Kubernetes with an example	[] Low [] Med [] High
7	How do you handle application configuration in K8s?	[] Low [] Med [] High
8	What are Probes? Explain Liveness vs Readiness vs Startup	[] Low [] Med [] High
9	How does Horizontal Pod Autoscaler work?	[] Low [] Med [] High
10	What is a CRD and an Operator?	[] Low [] Med [] High
11	How do you troubleshoot CrashLoopBackOff?	[] Low [] Med [] High
12	Explain Network Policies	[] Low [] Med [] High
13	What are Pod Disruption Budgets?	[] Low [] Med [] High
14	How do you do rolling updates and rollbacks?	[] Low [] Med [] High
15	How do you manage secrets securely in K8s?	[] Low [] Med [] High

5. CI/CD (10 Questions)

#	Question	Confidence
1	What is the difference between CI, CD, and CD?	[] Low [] Med [] High
2	Design a CI/CD pipeline for a microservices application	[] Low [] Med [] High
3	What is pipeline as code?	[] Low [] Med [] High

4

Jenkins declarative vs scripted pipeline

[ ] Low [ ] Med [ ] High

5

How do you handle secrets in CI/CD pipelines?

[ ] Low [ ] Med [ ] High

6

What is GitOps and how does ArgoCD implement it?

[ ] Low [ ] Med [ ] High

7

Explain blue-green vs canary vs rolling deployments

[ ] Low [ ] Med [ ] High

8

How do you implement automated rollback?

[ ] Low [ ] Med [ ] High

9

What are Jenkins shared libraries?

[ ] Low [ ] Med [ ] High

10

How do you set up GitHub Actions for a CI/CD workflow?

[ ] Low [ ] Med [ ] High

6. Terraform & IaC (10 Questions)

#	Question	Confidence
1	What is Terraform state and how do you manage it?	[ ] Low [ ] Med [ ] High
2	What are Terraform modules and why use them?	[ ] Low [ ] Med [ ] High
3	How do you handle state locking?	[ ] Low [ ] Med [ ] High
4	What is terraform import and when is it used?	[ ] Low [ ] Med [ ] High
5	Count vs for_each when to use which?	[ ] Low [ ] Med [ ] High
6	How do you manage different environments (dev/staging/prod)?	[ ] Low [ ] Med [ ] High
7	What are data sources in Terraform?	[ ] Low [ ] Med [ ] High
8	How do you handle drift detection?	[ ] Low [ ] Med [ ] High
9	Terraform vs CloudFormation vs Pulumi	[ ] Low [ ] Med [ ] High
10	How do you implement Terraform in a CI/CD pipeline?	[ ] Low [ ] Med [ ] High

7. Cloud (AWS/Azure) (10 Questions)

#	Question	Confidence
1	Explain VPC architecture and components	[ ] Low [ ] Med [ ] High
2	Security Groups vs NACLs	[ ] Low [ ] Med [ ] High
3	How does IAM role assumption work?	[ ] Low [ ] Med [ ] High
4	Compare EKS vs AKS	[ ] Low [ ] Med [ ] High
5	What is the shared responsibility model?	[ ] Low [ ] Med [ ] High
6	How do you implement cost optimization?	[ ] Low [ ] Med [ ] High
7	Explain multi-region deployment strategies	[ ] Low [ ] Med [ ] High
8	How do you handle disaster recovery?	[ ] Low [ ] Med [ ] High
9	What is cross-account access in AWS?	[ ] Low [ ] Med [ ] High
10	Serverless vs containerized when to choose which?	[ ] Low [ ] Med [ ] High

8. Monitoring & Observability (5 Questions)

#	Question	Confidence
1	What are the three pillars of observability?	[ ] Low [ ] Med [ ] High
2	Explain Prometheus + Grafana architecture	[ ] Low [ ] Med [ ] High
3	What are SLIs, SLOs, and SLAs? Give examples	[ ] Low [ ] Med [ ] High
4	How do you implement centralized logging?	[ ] Low [ ] Med [ ] High
5	What are the Golden Signals?	[ ] Low [ ] Med [ ] High

9. AI & DevOps (5 Questions)

#	Question	Confidence
1	How do you use AI tools in your DevOps workflow?	[ ] Low [ ] Med [ ] High
2	What is AIOps?	[ ] Low [ ] Med [ ] High
3	How can AI improve CI/CD pipelines?	[ ] Low [ ] Med [ ] High
4	What is prompt engineering for DevOps?	[ ] Low [ ] Med [ ] High
5	How do you ensure AI-generated IaC is secure?	[ ] Low [ ] Med [ ] High

10. Scenario-Based Questions

#	Scenario	My Approach
1	Production is down. Users report 500 errors. Walk me through your troubleshooting process.	
2	A deployment introduced a bug. How do you rollback safely?	
3	The Kubernetes cluster is running out of resources. How do you investigate and fix?	
4	A junior engineer accidentally deleted the Terraform state file. What do you do?	

5

You need to migrate a monolithic application to microservices. How do you approach it?

6

A CI/CD pipeline takes 45 minutes. How do you optimize it?

7

You discover a critical vulnerability in a base image used by 20+ services. How do you handle it?

8

The monitoring system shows increasing latency over the past week. How do you investigate?

9

A team wants to deploy their first application to Kubernetes. How do you guide them?

10

You're asked to set up a disaster recovery plan. What components do you include?

11

Your app relies on a 3rd-party API that went completely offline, bringing your whole app down. How would you have architected this differently?

Circuit Breaker + Cache + Queue + Graceful Degradation

11. Behavioral Questions (STAR Method)

#	Question	Your STAR Answer
1	Tell me about a time you handled a production outage	S: T: A: R:
2	Describe a situation where you disagreed with a team member	S: T: A: R:
3	How do you prioritize when multiple urgent tasks arise?	S: T: A: R:
4	Tell me about a project you're most proud of	S: T: A: R:
5	Describe how you learned a new technology quickly	S: T: A: R:

Confidence Tracker

Topic	Low	Medium	High	Ready?
Linux & Shell				
Git				
Docker				
Kubernetes				
CI/CD				
Terraform				
Cloud				
Monitoring				
AI/DevOps				
Scenarios				
Behavioral				

Tip: Use Claude to simulate interviews. Prompt: "You are a senior DevOps interviewer at a top tech company. Interview me for 30 minutes covering [topic]. Start with easy questions and increase difficulty. Score my answers."

---

12. Scenario Deep-Dives (Worked Answers)

Scenario 11 3rd-Party API Outage: How to Architect for Resilience

Question: Your application relies on an external 3rd-party API that just went completely offline, bringing your whole application down. How would you have architected this differently?

Root Cause

Tight coupling the app has a single point of failure (SPOF) on an external dependency with no fallback, cache, or circuit protection.

Resilience Architecture Key Patterns

1. Circuit Breaker Pattern

- Use libraries: Resilience4j (Java), Polly (.NET), opossum (Node.js)
- When failures exceed threshold circuit opens requests short-circuit immediately
- Periodically probes backend half-open state auto-recovery
- Prevents thundering-herd problem on recovery

2. Caching Layer (Stale-While-Revalidate)

- Cache last-known-good responses in Redis or CDN
- Serve stale data with Cache-Control: stale-if-error headers
- Label in UI: "Data as of 2 hours ago" transparency over silent failure

3. Async / Queue-Based Decoupling

- Never call the 3rd-party API synchronously in the request path
- Push work to a message queue (SQS, Azure Service Bus, RabbitMQ)
- Workers consume from the queue; if API is down, messages queue up safely
- App stays responsive; backlog is processed on recovery

4. Graceful Degradation

- Distinguish must-have vs nice-to-have features
- If API powers "recommendations" hide the widget, don't crash the page
- Return partial responses with clear status indicators (503 header)

5. Secondary Provider / Fallback

- Have a secondary vendor on standby (e.g., two payment gateways, two SMS providers)
- Auto-failover via weighted routing or DNS switch (Route 53 / Azure Traffic Manager)

6. Proactive Observability

- Add a synthetic monitor (Datadog, Dynatrace) polling the 3rd-party endpoint every 60s
- Alert on latency spike before users notice
- GET /health endpoint exposes upstream dependency status

7. Vendor Risk & SLA Assessment

- 99.9% SLA = ~8.7 hrs/year downtime acceptable?
- Negotiate backup endpoints or credits; evaluate if function can be internalized

One-Liner Summary (say this in interviews)

"The fix is to never couple your app's availability to a 3rd-party's availability. I'd use a circuit breaker for fault isolation, a cache for continuity, a queue for async decoupling, and graceful degradation so core flows stay alive even when the dependency is dead."

Before vs After

Aspect	Before (Bad)	After (Resilient)
Coupling	Synchronous, tight	Async, loosely coupled
Failure mode	Total app outage	Degraded but functional
Recovery	Manual intervention	Auto-recovery via circuit breaker
Visibility	None	Proactive alerting + health checks
User impact	100% down	Partial feature unavailability